



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Operating System for a High-Throughput Banking System

CSA0407 – OPERATING SYSTEMS

GUIDED BY

DR. J. VASAVI

DONE BY

JABARAJ T (192511187)

Problem And Requirements

Challenges in Banking Systems

- Thousands of concurrent transactions
- Fast response requirements
- Secure resource access
- Reliable transaction processing
- Network-based communication

OS Requirements

- Concurrency
- Reliability
- Security

OS Suitable for Banking

- **Objective:** Select suitable OS types, explain the OS structure, and understand the role of system calls in transaction processing.
- A banking system handles thousands of transactions simultaneously. The OS must support concurrency, reliability, and security.

Suitable approach

- **Multi-user OS:** supports multiple authenticated users and processes concurrently across customers, employees, ATMs, and services.
- **Multitasking / multiprocessing OS:** executes multiple processes or threads efficiently for concurrent transaction workloads.
- **Network / server OS:** provides networking, security, and shared services across applications, ATMs, branches, and client systems.
- **Combined model:** multi-user + multitasking/multiprocessing + network/server capabilities.

Layered OS Structure Creates a Clear System-Call Boundary

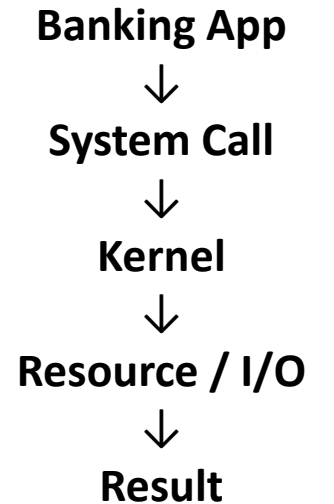
1. **Banking applications:** ATM, mobile banking, web banking, and branch applications.
2. **System call interface:** controlled interface between applications and the kernel.
3. **Kernel services:** process management, memory management, file system, network management, and security.
4. **Hardware / devices:** CPU, RAM, storage, and network interfaces.

The layered structure separates applications from privileged operations, provides controlled resource management, improves protection, and organizes the system.

System Calls Connect Transactions to Protected OS Services

- **socket() / connect() / accept():** network communication.
- **send() / recv():** exchange transaction data.
- **open() / read() / write():** access files and logs.
- **fsync():** request data to be flushed to storage.

1. Client: ATM or mobile app.
2. Banking application validates and processes the transaction.
3. System calls and the kernel manage resources and I/O.
4. Storage or database persists the transaction result.
5. Response returns to the customer.



Reliable Banking Transactions Follow Five Controlled Steps

1. **01 — REQUEST:** customer initiates a transaction.
2. **02 — PROCESS:** banking application validates the request.
3. **03 — KERNEL:** system calls request protected OS services.
4. **04 — PERSIST:** transaction data is stored.
5. **05 — RESPONSE:** result is returned to the customer.

Key takeaways

- Multi-user and multiprocessing capabilities support concurrent banking workloads.
- Structured OS architecture manages resources and protects privileged operations.
- System calls provide the controlled bridge between applications and kernel services.
- Together, these mechanisms support reliable, secure, and scalable transaction processing.

Thank You

Presentation photo

